National University of Computer and Emerging Sciences

Lahore Campus

Data Structures Lab (CL2001)

BCS-3C

Midterm Exam

Date: October 13, 2026

Total Time (Hrs): 2

Total Marks: 25

Total Questions: 2

Instructor

Mr. Jawad Khan

Submission Path

Windows + R: \\Exam\ Exam\Fall 2026 Midterm Exam\ DS Lab\ [your section]\ Submissions

Instructions:

- Duration of exam is 2 hours. An additional 15 minutes have been allocated for proper submission of your files. Therefore, stop attempting the exam once the 2 hour duration is over and use the remaining time to carefully submit your file.
- Make a .cpp file for each question and name it using your roll number e.g. 25L-1234-q1.cpp
- Place all the exam (.cpp) files in a single folder and compress the folder into a ZIP file named using your roll number e.g. 25L-1234.zip and upload the ZIP file to the designated submission folder.
- After submitting, verify your zip file is visible in the submission folder of your section, and check that a file size is shown next to it (to ensure it is not 0 KB file which indicates the file is empty or corrupted).
- To ensure everything is properly submitted, you may download the uploaded ZIP file and check its contents. It is your responsibility to save and submit all required files in the correct format and ensure that the submitted files are complete and not corrupted.
- Submit your files inside your section's submissions folder

Attempt all the questions.

CLO #1,2

Q1: Recursive Maze Trail Backtracking [15 marks]

Instructions:

- You may not use any STL containers (e.g. vector, stack, map) or built-in string manipulation functions.
- Use dynamic memory wherever appropriate. Global variables not allowed.
- Implement the Stack yourself using a Singly Linked List.

Problem:

Implement a Stack data structure using a Singly Linked List. Write a recursive function solve_maze_recursive(...) that finds a path from the start coordinate (0,0) to the exit coordinate (N-1, M-1) in a 2D grid matrix (0 = Open Path, 1 = Wall).

- **Requirement:** As recursion explores a valid path, push the current cell coordinates (row, col) onto your Linked List Stack. If a path hits a dead end, pop from your stack upon returning. Print the final path coordinates from your stack once the exit is reached.

Example:

- Input Grid:

```
[0, 1, 0, 0]
[0, 0, 0, 1]
[1, 1, 0, 0]
```

- Start: (0, 0), Exit: (2, 3)
- Expected Output Path (Stack Content top-to-bottom or start-to-end):

```
Path Found: (0,0) -> (1,0) -> (1,1) -> (1,2) -> (2,2) -> (2,3)
```

CLO #2

Q2: Daily Temperatures (Days to Warmer Weather) [10 marks]

- **Task:** Given an array of daily temperatures, return an array answer such that answer[i] is the number of days you have to wait after the ith day to get a warmer temperature. If there is no future day with a warmer temperature, set answer[i] = 0.
- **Requirement:** Store temperature indices in your Linked List Stack (Implemented in Q1) in a monotonically decreasing sequence.
- **Time Constraint:** Must operate in **O(N) time complexity**.

Example:

- **Input Temperatures:** [73, 74, 75, 71, 69, 72, 76, 73]
- **Calculations:**
  - Day 0 (73) > Warmer day is Day 1 (74) > Wait 1 day.
  - Day 2 (75) > Warmer day is Day 6 (76) > Wait 4 days.
- **Expected Output:** [1, 1, 4, 2, 1, 1, 0, 0]

----------Good luck! ☺----------